QA Report for CHESS MANIA!

Mark Starzynski
André Vieira Ribeiro

FS21

1. Introduction

The usefulness of QA procedures and the necessity of those, were clear from the very start. It became more and more obvious as time passed by why quality assurance has such a vital part in software development. As we have learned, working on a project with thousands of lines can get quite challenging, if there are no QA measures in place.

In the following section, we will depict our software quality assurance methods and analyse their implementation in our project. At the end, we will evaluate the overall success of our implementations and contemplate on what could have been done to further improve the quality of our code.

2. Results

1. Communication

There is no doubt that communication is the basis of a project. In times like these where meetings have to be held online and teams are unable to meet in person and create a productive working environment, we had to make sure to always stay in touch.

After Milestone 2 our group consisted of only two members, so we needed to constantly communicate with each other to ensure sufficient progress. Not so much for the implementation of code, but rather more for the project management. In the first week we used to communicate primarily through WhatsApp but then moved on to Slack because of its features and better overall experience. We also started to hold long Zoom Meetings after every Milestone to assign tasks and responsibilities to each other. To make that task easier and more productive we started using Trello, a team task manager which allows us to assign tasks and follow their progress. With Trello we could track things on-the-fly and always know what to do next. Also each weekend before the deadline we introduced prolonged coding sessions. This turned out

to be a very successful strategy. By constantly enforcing these measures we were able to simplify and speed up the progress considerably.

2. Log4j2

We have persistently implemented log4j2 in our try and catch blocks for better error handling and exception catching. We also instructed the plugin to create log files in order to keep track of all the errors. But besides all our efforts to turn this into a useful tool, we didn't really find much practicality in it. Before implementing log4j2 we were using simple System.out.println() methods that can basically do everything the log4j2 are doing now. Maybe if it weren't for the immense time pressure we could have found its possibly hidden usefulness. Log4j2 in our project is mostly used in the client and server classes, which are the classes most prone to errors.

3. Unit Tests

Up until this point, the only experience we had regarding Unit Tests came from our previous programming lecture "Erweiterte Grundlagen der Programmierung". Since this was our first time actually implementing serious Unit Tests, we made the small mistake of testing too many conditions in one single test method. As soon as this issue was pointed out to us, we fixed the Unit Tests and also added illegal chess moves, in order to enhance the usefulness of our Unit Tests. In fact the Unit Tests turned out to be very helpful at finding bugs in the game logic, as for example the movement of the queen. The queen piece couldn't move more than one square diagonally to the left. We noticed that the move was being miscalculated in the complex game logic and yet we were able to very quickly fix the issue.

There are currently a total of 247 tests including 469 assertions. All game relevant features like the chess pieces, the chessboard, the single squares, the player and the game itself are thoroughly tested and work as intended.

4. Metrics

We used the IntelliJ IDEA plugin "Metrics Reloaded", as suggested during the lectures, to track the quantitative data regarding our code throughout this project. This data can be found within the Metrics-ChessMania.xlsb file or within the Metrics-ChessMania.pdf file.

In Figure 1, the percentage of documented code is graphed, whereby the x-axis shows the tracked dates and the y-axis shows the percentage of documented classes, methods and fields.

In the beginning of our project, we tended to forget to document our code using javadoc. It didn't seem to be a problem at first but with the rapidly growing quantity of code, it started to become difficult to understand each other's code and sometimes even our own. We didn't set any specific goal, but we agreed on documenting at least every major method and\or function to ensure that they can be understood by anyone at any time. A big increase of javadoc usage can be seen around April 6th. This was when the game logic was being implemented and demanded an extensive usage of javadoc to keep everything well documented and understandable. Because of the growing amount of classes, it wasn't always possible to keep up with the documentation. To catch up on the missing Javadoc documentation we went over all our code before every milestone deadline. Currently, the class coverage is at about 80%, while the method coverage hovers at about 50%. The field coverage is in comparison very low, only reaching 13.68%.

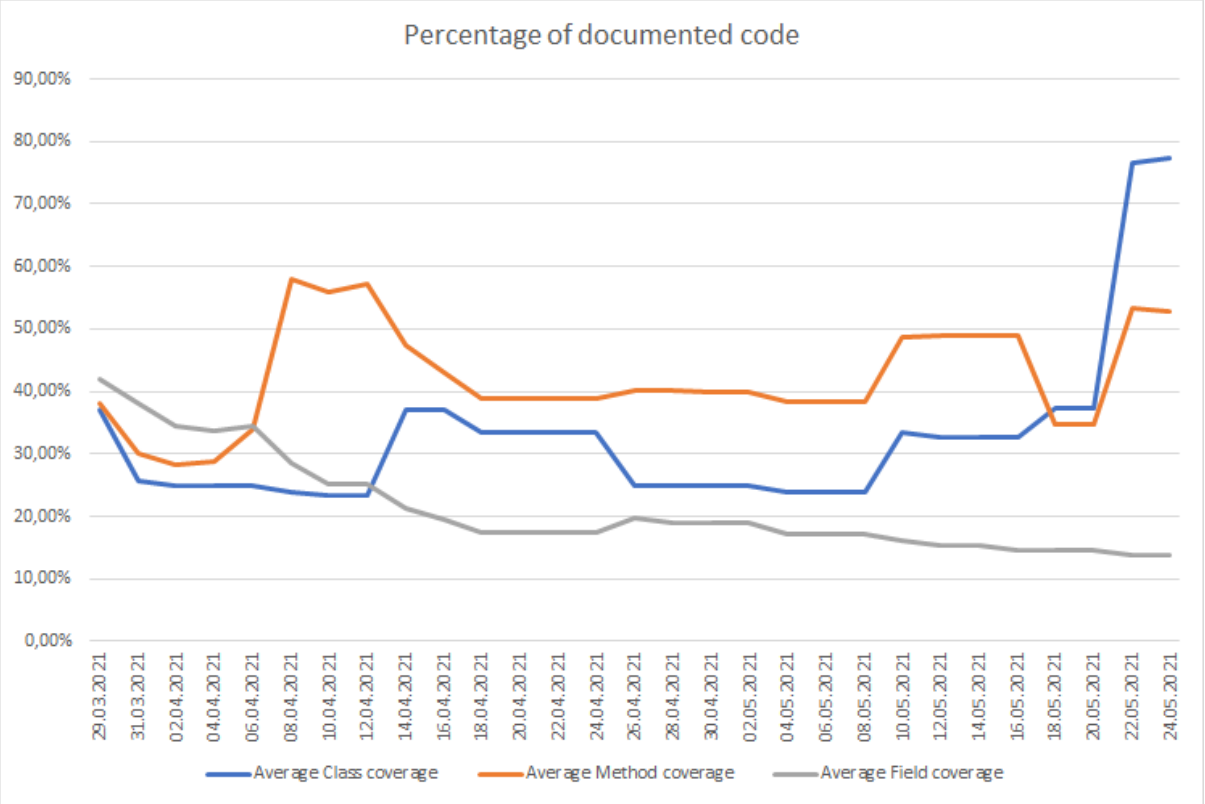


Figure 1: The percentage of documented code. The x-axis shows the tracked dates and the y-axis shows the percentage of documented classes, methods and fields.

Figure 2 shows the cyclomatic complexity in relation to the total lines of code. The amount of code has increased in a rather linear way, while the total cyclomatic complexity has increased minimally. The current average cyclomatic complexity is at one of its lowest values ever, at 3.43. Because of the complex nature of chess, our

project does have methods exceeding a cyclomatic complexity of 90. If we had more time and more manpower, we would address this issue.

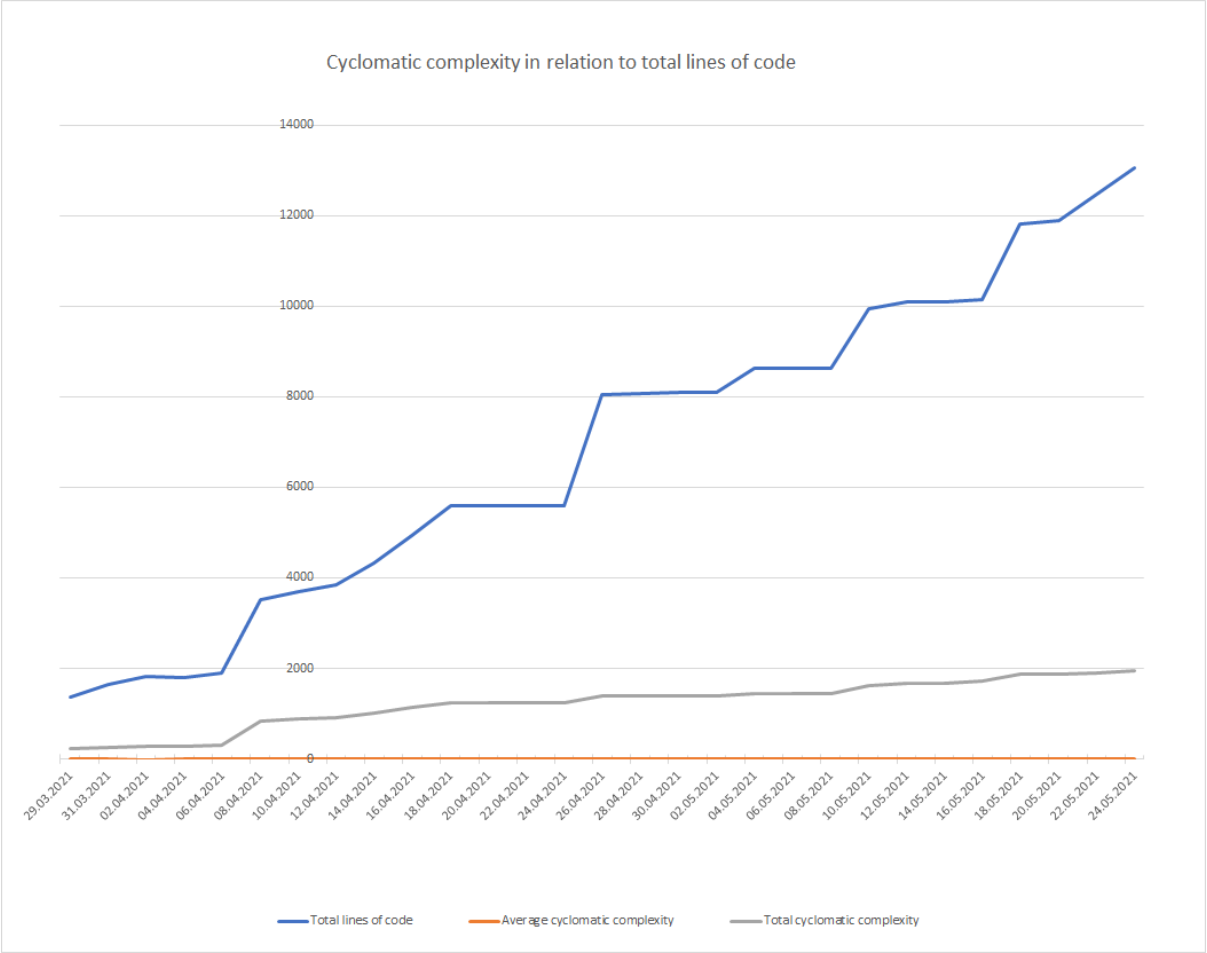


Figure 2: The cyclomatic complexity in relation to the total lines of code.

3. Conclusion

As expected, starting with a project of this scope, it is crucial to have a well thought out schedule and very carefully plan ahead each step as meticulously as possible. Set project time frames always seem to prolong, especially as the complexity of the project grows with time, so it is important to have generous buffer zones in between important milestones to ensure the goals are met at the proper time.

Right from the start we were met with additional challenges as well. It seemed that even though the project is extremely fun and enjoyable, not all human resource capabilities could be activated and the success of the project seemed doomed. Thankfully, we didn’t give up and started to develop

solutions to problems step-by-step, yet with foresight. The tools that have been set up from week one, but never used (i.e. Trello and Slack), were prescribed as mandatory. Sacrifices in regards to other lectures could not be avoided at the start, but luckily receded as the project progressed.

Along the way we managed to write more than 13000 lines of code across 43 classes, with an average class coverage of 77,27%. Because of the complex nature of chess, we do have a higher cyclomatic complexity. But we were able to meaningfully reduce that number and simplify our code, the more code we wrote. Although it isn't perfect, it was definitely enough to achieve a good endresult. And in the end it showed us the importance of keeping track of quantitative data and how it can influence a project.

Overall we can't really say that we didn't try our best given the situation. A lot of features we wanted to include with this game didn't make it to the final version, we had to just suck it up and reduce our workload, otherwise the workload would have been impossible to handle for two. Managing your resources and time is quite important in a project, but we couldn't really practice those skills as efficiently as other, full teams might have. The dynamic of a duo developer team is quite different from that of four.

For future project endeavours we certainly learned that you can't always control who you have to work with. You just have to make the best out of it. In the end, we are proud of what we have accomplished.